



프로젝트 디샤

2026년 5월 2일

팀: 내MOM대로

팀원 : 강우석(팀장), 최민섭, 배상규, 김지율
동양미래대학교 컴퓨터 소프트웨어 공학과
3학년 YA반 졸업작품 설계서

프로젝트 개요	1
1. 시스템의 정의	1
2. 개발 배경 및 필요성	1
3. 활용 방안	2
4. 개발 목표	2
5. 제약 사항	2
구조 설계	2
1. 시스템 구성 및 주요 기능	2
2. 프로젝트 기능 분할 (트리 구조도)	3
3. 사용자 특성	4
4. 팀원 역할 분담	5
상세 설계	5
1. 사용자 인터페이스 설계 (UI/UX)	5
1. 인터페이스 양식 및 디자인 시스템	5
2. 주요 화면 설계	5
3. 자료구조 정의 (데이터 베이스 설계 -ERD)	9
🔗 Dicha Coffee 데이터베이스 연결 흐름도	10
🏃 Scene 1. 상규님이 쇼핑몰에 가입하고 취향을 테스트합니다.	11
👁 Scene 2. 상품을 구경하고 장바구니에 담습니다.	11
💳 Scene 3. 장바구니에 있는 물건을 결제합니다. (가장 중요!)	11
📝 Scene 4. 커피를 마시고 후기를 남깁니다. (또는 구독/예약을 합니다)	12
6. 모듈 명세서 (소기능)	12
7. 통신 설계	12
8. 보안 및 데이터 보호	12
프로토타입 시스템 구축 계획 및 소개	12
1. 예상 데모시스템의 사용 시나리오	12
시나리오 1 : 일반 고객의 맞춤형 원두 구매 흐름	12
시나리오 2: 관리자의 주문 처리 흐름	13
2. 구축할 시스템의 구성도	13
1. 클라이언트 사이드 (프론트 엔드)	13
2. 서버 사이드 (백엔드)	13
3. 데이터 베이스	13
3. 구현할 기능의 우선순위	13
1순위	13
2순위	14
5. 기타	14
1. 프로토타입 구현 시 예상 문제점 및 해결 방안	14
프론트 엔드 - 백엔드 연동 오류 (CORS 문제)	14
커스텀 옵션에 따른 데이터 무결성 유지	14
2. 앞으로의 발전 방향 (2차 고도화 및 상용화)	14
완벽한 O2O 생태계 구축	14
실제 상용 서비스 배포 및 인프라 고도화	15

프로젝트 개요

1. 시스템의 정의

본 프로젝트는 기존 오프라인으로 운영 중인 로스터리 카페의 브랜드 경험을 온라인으로 확장하는 O2O(Online to Offline) 기반 커스텀 원두 이커머스 플랫폼 'DICHA Coffee'의 구축을 의미한다.

단순한 기성품 판매를 넘어, 사용자의 취향을 분석하여 최적의 원두를 추천하는 큐레이션 서비스와 세분화된 커스텀 로스팅(원산지, 로스팅 정도, 분쇄도, 용량 선택) 주문 시스템을 제공하는 웹 어플리케이션이다.

2. 개발 배경 및 필요성

오프라인 매장의 한계 극복 및 상업적 확장 : 기존 부모님이 운영하시는 로스팅 카페의 상권 한계를 벗어나, 운영을 자동화하고 전국적인 고객층을 확보하기 위한 맞춤형 온라인 커머스 창구가 필요하다.

초보자 친화적인 커피 큐레이션의 부재 : 기존 원두 쇼핑몰들은 신선도 보장, 깔끔한 디자인 등 장점이 있으나, 커피에 대한 전문적인 설명이 주를 이루어 일반 소비자가 자신의 취향에 맞는 원두를 선택하기 어렵다는 단점이 존재한다.

사용자 경험(UX) 개선 필요 : 이를 보완하기 위해 '커피 취향 테스트', '취향에 맞는 원두 추천 서비스'를 도입하여, 커피 입문자도 쉽고 직관적으로 자신에게 맞는 원두를 찾을 수 있는 시스템이 요구된다.'

3. 활용 방안

O2O(Online to Offline) 연계 마케팅 :

Online => Offline : 웹사이트에서 제공되는 브랜드 스토리와 원두 큐레이션 경험을 통해 오프라인 매장 방문 및 바리스타/핸드드립 클래스 수강 유도

Offline => Online : 오프라인 매장을 방문한 고객이 온라인 플랫폼을 통해 원두 정기 구독, 재구매, 선물세트 등 구매를 쉽게 진행 할 수 있도록 연결한다.

데이터 기반 맞춤형 주문 :

사용자가 선택한 커스텀 로스팅 옵션을 마이페이지에 저장하여 재구매 편의성을 높이고, 관리자에게는 해당 옵션에 맞춘 '로스팅 작업 지시서'가 자동 생성되어 매장 운영 효율성을 극대화 한다.

4. 개발 목표

기술적 목표: Spring Boot (백엔드), React(프론트엔드), MySQL(데이터베이스), Amazon AWS(클라우드 서버) 등을 활용하여 프론트엔드와 백엔드가 분리된 최신 RESTful API 아키텍처 기반의 웹 서비스를 설계하고 구현한다.

기능적 목표: 회원가입/로그인, 장바구니, 결제 등 기본적인 이커머스 기능을 완벽하게 구현하며, 최우선 핵심 기능인 '커스텀 로스팅 주문' 과 '커피 취향 테스트'를 성공적으로 서비스에 통합한다.

5. 제약 사항

본 시스템은 학부 졸업 작품 이자 1차 상용 배포를 목표로 하므로, 1차 개발 범위에서는 기본적인 커머스 기능 과 커스텀 로스팅, 취향 테스트에 집중한다.

구독 서비스 및 예약 시스템, 소셜 로그인 등 부가적인 심화 기능은 2차 개발 목표로 이관하여 핵심 기능의 완성도와 안정성을 우선 확보한다.

구조 설계

1. 시스템 구성 및 주요 기능

본 시스템은 사용자가 웹 브라우저를 통해 접속하는 프론트 엔드 (React) 화면과, 비즈니스 로직 및 데이터를 처리하는 백엔드(Spring Boot API + My SQL) 서버로 완벽히 분리된 구조를 가진다. 주요 기능은 다음과 같이 요약한다.

커스텀 주문 시스템 : 생두 원산지, 로스팅 강도, 분쇄도, 용량을 조합하는 단계별 상품 선택 및 장바구니 기능

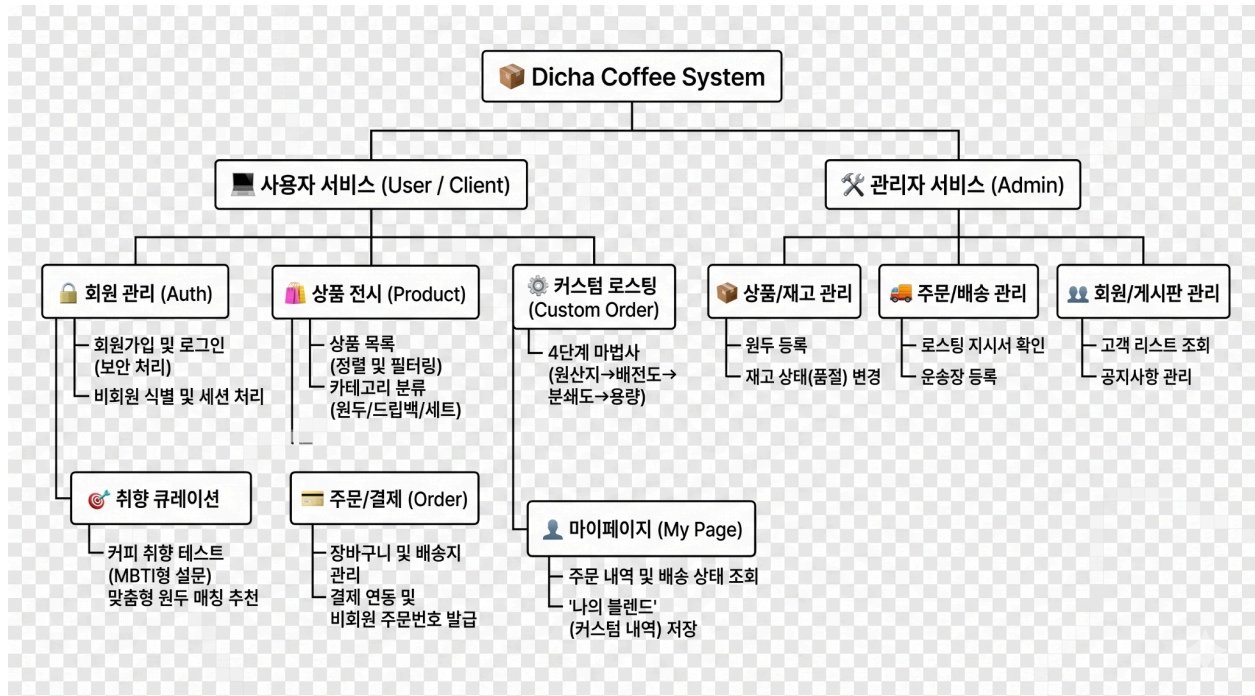
큐레이션 시스템 : 산미, 바디감, 추출 방식 등 사용자의 취향을 설문하여 적합한 원두를 매칭해주는 추천 알고리즘

회원 및 커머스 코어 : 회원가입/로그인(보안 적용), 비회원 주문, 배송지 관리, 결제 정보 연동 및 주문 내역 조회

관리자 (admin) 시스템 : 상품 등록, 재고 상태 변경, 주문 확인 및 '로스팅 작업 지시서' 확인, 배송 상태 업데이트

2. 프로젝트 기능 분할 (트리 구조도)

전체 시스템을 모듈별, 소기능별로 분할한 트리구조는 다음과 같다.



📦 Dicha Coffee System

- └─ 🖥️ 사용자 서비스 (User / Client)
 - | └─ 🔑 회원 관리 (Auth)
 - | | └─ 회원가입 및 로그인 (보안 처리)
 - | | └─ 비회원 식별 및 세션 처리
 - | └─ 🛒 상품 전시 (Product)
 - | | └─ 상품 목록 (정렬 및 필터링)
 - | | └─ 카테고리 분류 (원두/드립백/세트)
 - | └─ ⚙️ 커스텀 로스팅 (Custom Order)
 - | | └─ 4단계 마법사 (원산지→배전도→분쇄도→용량)
 - | └─ 🎯 취향 큐레이션 (Curation)
 - | | └─ 커피 취향 테스트 (MBTI형 설문)
 - | | └─ 맞춤형 원두 매칭 추천
 - | └─ 📄 주문/결제 (Order)
 - | | └─ 장바구니 및 배송지 관리
 - | | └─ 결제 연동 및 비회원 주문번호 발급
 - | └─ 👤 마이페이지 (My Page)
 - | | └─ 주문 내역 및 배송 상태 조회
 - | | └─ '나의 블렌드' (커스텀 내역) 저장
- └─ 🛠️ 관리자 서비스 (Admin)
 - └─ 📦 상품/재고 관리 : 원두 등록, 재고 상태(품질) 변경
 - └─ 🚚 주문/배송 관리 : 로스팅 지시서 확인, 운송장 등록
 - └─ 👥 회원/게시판 관리 : 고객 리스트 조회, 공지사항 관리

3. 사용자 특성

본 시스템을 이용할 주요 사용자 층은 다음과 같이 분류되며, 각 타겟에 맞춘 UI/UX를 제공한다.

초보/입문자 : 원두 종류를 잘 모르는 고객으로, '커피 취향 테스트'와 직관적인 추천 리스트를 주로 이용한다.

홈 카페족 (2030) : 집에서 직접 커피를 내려 마시며, '커스텀 로스팅'과 '분쇄도 선택' 기능의 활용도가 높다.

선물 구매자 : 명절, 기념일 등을 위한 드립백 선물 세트 카테고리를 주요 이용한다.,

관리자 (로스터/카페 사장) : 모바일 또는 PC로 접속하여 접수된 커스텀 옵션을 확인하고 당일 로스팅 작업 및 배송 처리를 진행한다.

4. 팀원 역할 분담

프로젝트의 성공적인 완수를 위해 프론트 엔드 파트와 백엔드/데이터 아키텍처 파트로 역할을 분담했다..

강우석 (팀장) : 프론트엔드 & UX 파트

최민섭 : 프론트 엔드 & UX 파트

배상규 : 백엔드 & 데이터 아키텍처 파트

김지율 : 백엔드 & 데이터 아키텍처 파트

상세 설계

1. 사용자 인터페이스 설계 (UI/UX)

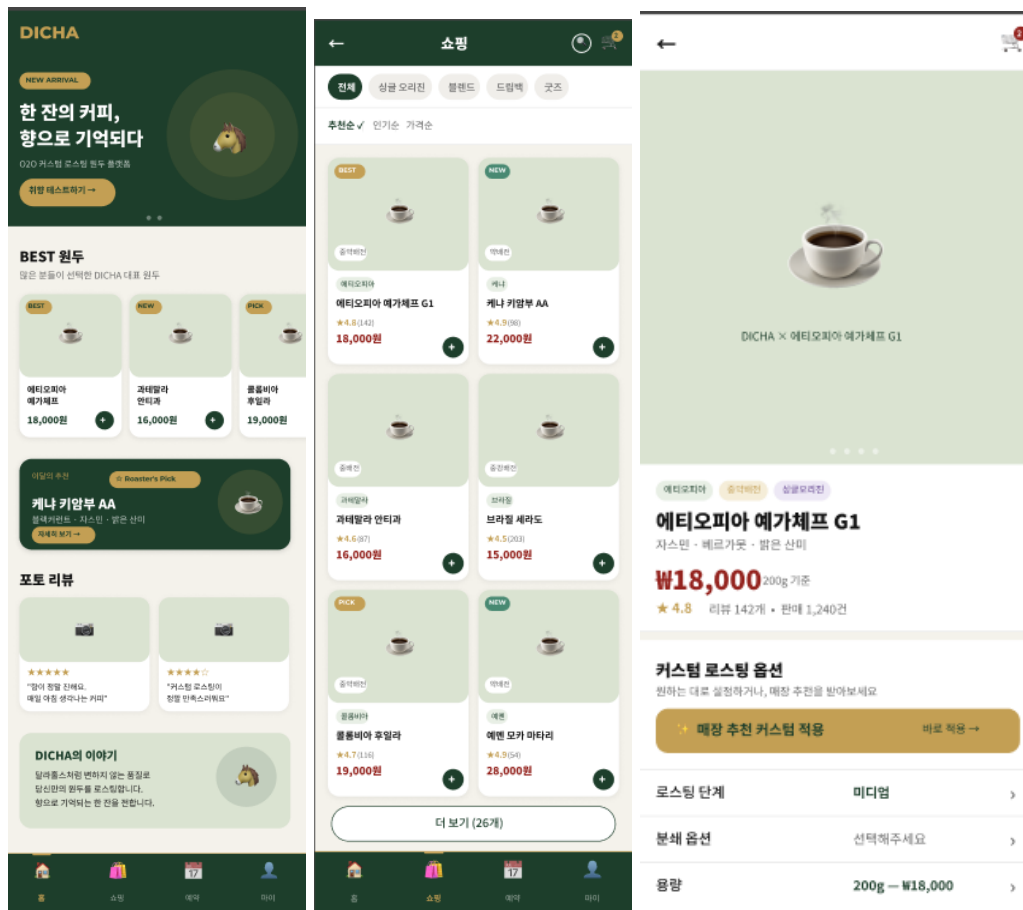
본 시스템의 인터페이스는 오프라인 로스터리 카페의 프리미엄 브랜드 이미지를 온라인으로 전달하기 위해, 모바일 사용성을 최우선으로 고려한 모바일 퍼스트 기반 반응형 웹으로 설계한다. 화면 설계 및 프로토타이핑은 Figma를 활용하여 진행한다.

1. 인터페이스 양식 및 디자인 시스템

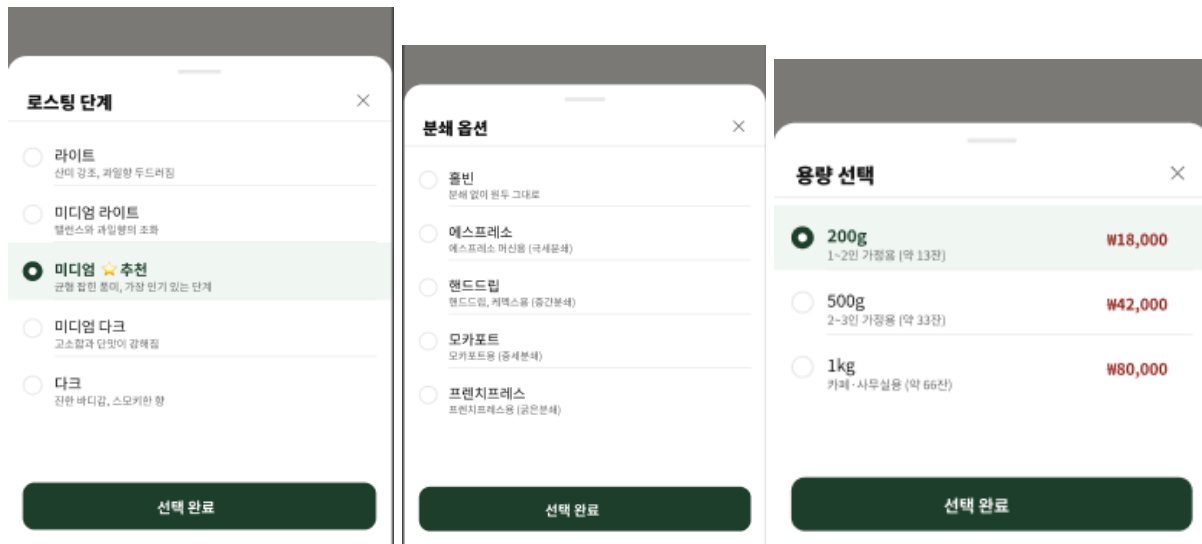
- 주요 색상: 메인 색상 #1D3E2B, #94231E, 배경 색상 #F4F2EB
- 타이포 그래피: 산세리프, 세리프체 혼용

2. 주요 화면 설계

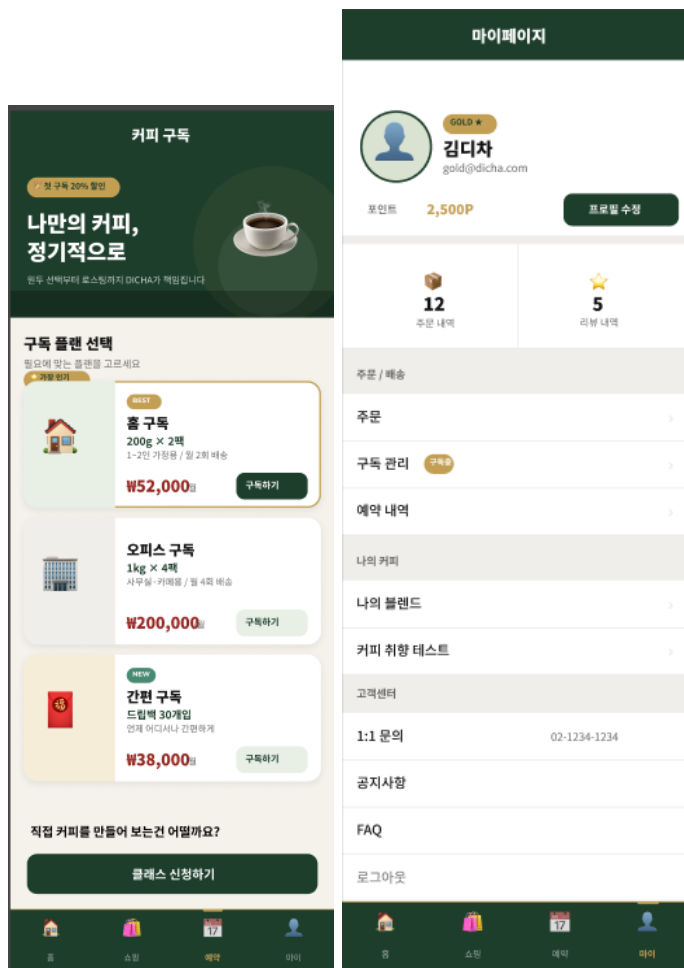
- 메인 화면, 쇼핑 화면, 상세 품목 화면



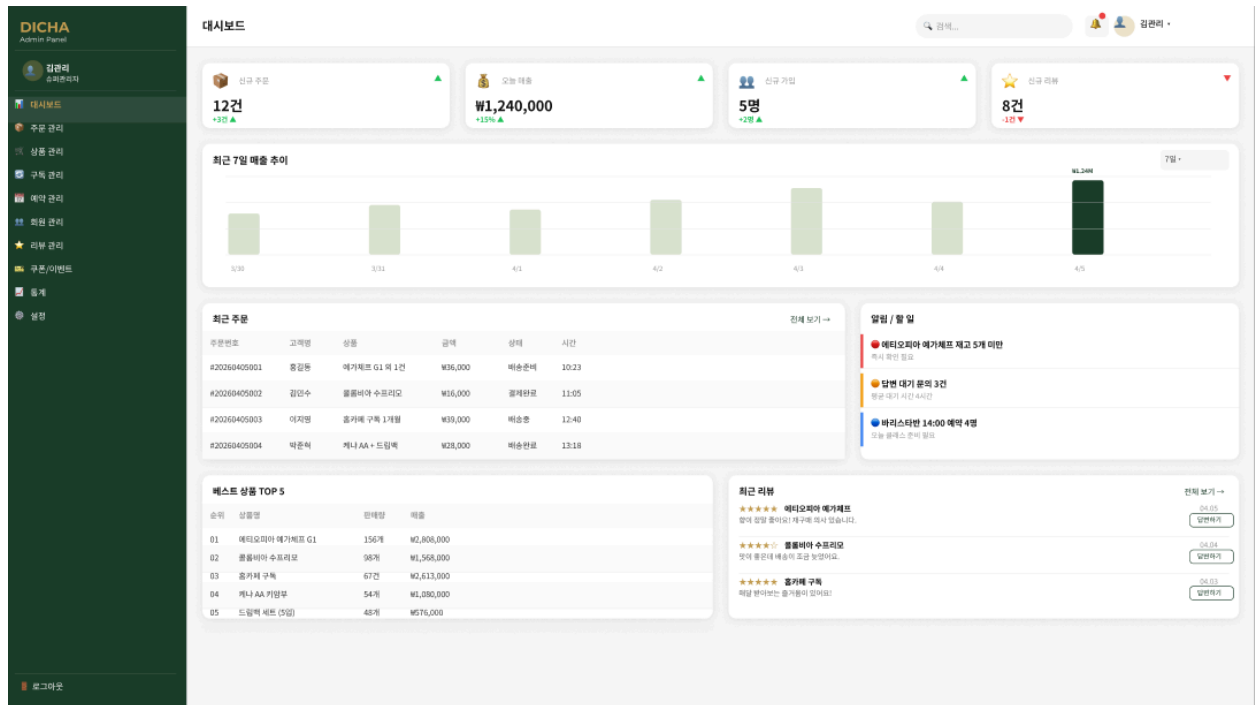
- 옵션 설정 화면



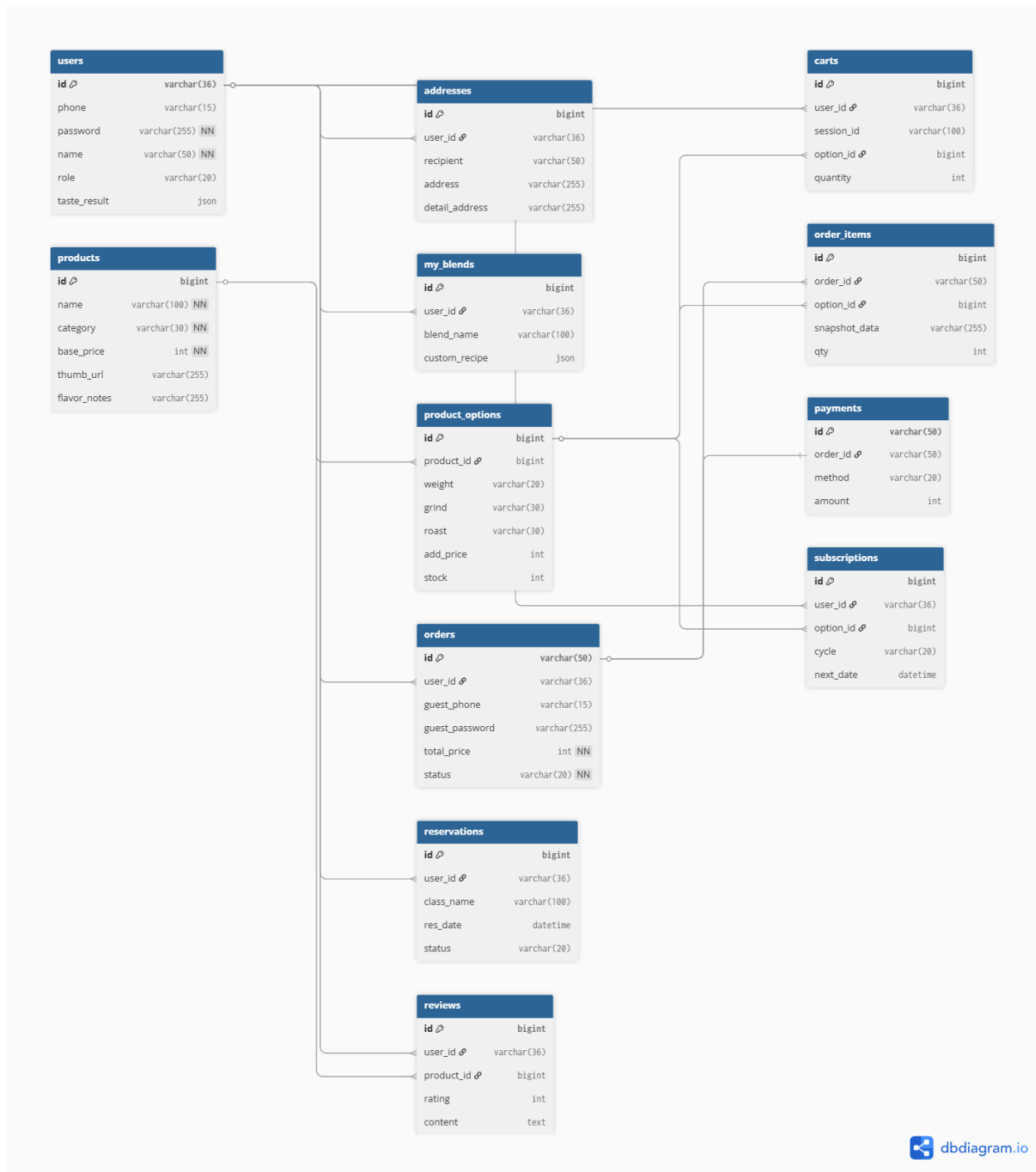
- 구독 화면, 마이페이지 화면



- 관리자 대시보드



3. 자료구조 정의 (데이터 베이스 설계 -ERD)



Dicha Coffee 데이터베이스 연결 흐름도

핵심 연결 원칙: (1)은 부모 테이블(1개), (N)은 자식 테이블(여러 개)을 의미합니다. 화살표(→)는 자식이 부모의 ID를 외래키(FK)로 가지고 있음을 뜻합니다.

Scene 1. 상규님이 쇼핑몰에 가입하고 취향을 테스트합니다.

- [회원] 1 : N [배송지]
 - `users` (1) ← (N) `addresses`
 - 상규님(`users`)은 집, 회사, 친구 집 등 여러 개의 배송지(`addresses`)를 등록할 수 있습니다. 배송지 테이블에 '상규님의 회원 ID(`user_id`)'가 적혀있어 누구의 주소인지 알 수 있습니다.
- [회원] 1 : N [나의 블렌드]
 - `users` (1) ← (N) `my_blends`
 - 상규님이 취향 테스트를 마치고 여러 가지 커스텀 원두 레시피를 '나의 블렌드'에 저장해 둡니다.

Scene 2. 상품을 구경하고 장바구니에 담습니다.

- [상품] 1 : N [상품 옵션]
 - `products` (1) ← (N) `product_options`
 - '디차 시그니처 원두'라는 상품(`products`) 하나에는 '200g/홀빈', '500g/핸드드립용' 등 수많은 조합의 옵션(`product_options`)이 존재합니다.
 - **중요:** 고객이 실제로 장바구니에 담고 결제하는 것은 껍데기인 '상품'이 아니라, 재고와 추가 금액이 있는 '상품 옵션'입니다.
- [회원] 1 : N [장바구니] N : 1 [상품 옵션]
 - `users` (1) ← (N) `carts` (N) → (1) `product_options`
 - 장바구니 테이블은 중간 다리 역할을 합니다. "상규님(`user_id`)이 시그니처 원두 500g 옵션(`option_id`)을 2개 담았다"라고 양쪽을 연결합니다.

Scene 3. 장바구니에 있는 물건을 결제합니다. (가장 중요!)

결제 순간, 장바구니에 있던 데이터가 '주문' 테이블로 복사되며 영수증이 만들어집니다. 영수증은 머리 부분(`orders`)과 세부 내역(`order_items`)으로 나뉩니다.

- [회원] 1 : N [주문 (영수증 머리)]
 - `users` (1) ← (N) `orders`
 - 상규님은 쇼핑몰에서 여러 번 주문을 할 수 있습니다. 주문 번호(ex: 260503-A1), 배송 상태, 총 결제 금액이 여기에 기록됩니다.
- [주문] 1 : N [주문 상세 (영수증 내역)]
 - `orders` (1) ← (N) `order_items`

- 하나의 주문(**orders**) 안에 "원두 1개, 드립백 2개"처럼 여러 개의 상세 품목(**order_items**)이 들어갑니다.
- **[상품 옵션] 1 : N [주문 상세]**
 - **product_options** (1) $\leftarrow$ (N) **order_items**
 - 주문 상세 내역이 정확히 어떤 상품 옵션(**option_id**)을 샀는지 가리킵니다.
- **[주문] 1 : 1 [결제 내역]**
 - **orders** (1) $\leftarrow$ (1) **payments**
 - 해당 주문에 대해 카드사에서 승인받은 결제 내역(영수증 번호, 승인 시각 등)을 1대1로 연결합니다.

 **Scene 4. 커피를 마시고 후기를 남깁니다. (또는 구독/예약을 합니다)**

4. **[회원] 1 : N [리뷰] N : 1 [상품]**
 - a. **users** (1) $\leftarrow$ (N) **reviews** (N) $\rightarrow$ (1) **products**
 - b. 리뷰 테이블은 누가(**user_id**) 어떤 상품(**product_id**)에 별점을 남겼는지 양쪽을 연결합니다.
5. **[회원] 1 : N [정기구독] N : 1 [상품 옵션]**
 - a. **users** (1) $\leftarrow$ (N) **subscriptions** (N) $\rightarrow$ (1) **product_options**
 - b. 누가 어떤 원두 옵션을 정기적으로 받기로 했는지 연결합니다.

6. 모듈 명세서 (소기능)

- Auth (인증/보안) 모듈
- Custom Order (커스텀 주문) 모듈
- Curation (취향 추천) 모듈

7. 통신 설계

- 아키텍처 : 클라이언트 (React) 와 서버(Spring Boot) 는 HTTP/HTTPS 프로토콜 기반의 RESTful API를 통해 통신한다.
- 데이터 교환 형식 : 모든 요청과 응답 데이터는 JSON 포맷을 사용한다.
- CORS(교차 출처 리소스 공유) 정책 : 프론트 개발 서버 및 상용 배포 도메인에서 들어오는 API 요청을 허용하기 위해 백엔드의 WebMvcConfigurer를 통해 명시적인 CORS 허용 규칙을 설정하여 연동 에러를 사전에 방지한다.

8. 보안 및 데이터 보호

- 비밀번호 암호화 : 전자상거래법 및 개인정보보호법에 의거하여, 사용자의 비밀번호는 DB 관리자도 알 수 없도록 스프링 시큐리티의 BCryptPasswordEncoder 를 적용해 단방향 해시 및 솔팅 처리하여 저장한다.

프로토타입 시스템 구축 계획 및 소개

1. 예상 데모시스템의 사용 시나리오

프로토타입 시연은 본 서비스의 핵심 차별화 요소인 '큐레이션'과 '커스텀 주문'이 자연스럽게 구매로 이어지는 퍼널을 보여주는 것에 집중한다.

시나리오 1 : 일반 고객의 맞춤형 원두 구매 흐름

1. 유입 및 탐색 : 사용자가 메인 페이지에 접속하여 커피 취향 테스트 배너를 클릭한다.
2. 취향 테스트 : 산미, 바디감, 평소 마시는 방식 등 문항에 답변하면 시스템이 사용자의 취향에 가장 적합한 원두 3종을 결과로 추천한다.
3. 커스텀 주문 : 추천받은 원두 중 하나를 선택한 후, '커스텀 로스팅으로 주문하기'를 클릭한다.
4. 상세 옵션 선택: 생두 원산지 확인 → 로스팅 단계 선택(슬라이더 바 조작) → 분쇄도(핸드드립용 등) 선택 → 용량(500g 등)을 차례대로 선택하며 실시간으로 변동되는 가격을 확인한다.
5. 주문 및 결제 : 장바구니에 상품을 담고, 비회원(또는 로그인) 상태로 배송지를 입력한 후 가상 결제를 진행하여 최종 주문 번호를 부여받는다.

시나리오 2: 관리자의 주문 처리 흐름

1. 주문 확인 : 관리자가 대시보드에 로그인하여 신규 접수된 커스텀 주문 내역을 확인한다.
2. 작업 지시서 확인 : 해당 주문의 '로스팅 단계'와 '분쇄도'가 요약된 '로스팅 작업 지시서' 데이터를 확인하여 오프라인 로스팅 작업을 시뮬레이션한다.
3. 배송 처리 : 작업 완료 후 주문 상태를 '배송 중'으로 변경한다.

2. 구축할 시스템의 구성도

프로토타입 시스템은 프론트엔드와 백엔드를 완전 분리하여 구축하며, 안정적인 데모 시연을 위해 클라우드 인프라를 활용한 구성을 계획한다. (단, 초기 개발 및 로컬 테스트 단계에서는 모두 Localhost 환경에서 구동한다.)

1. 클라이언트 사이드 (프론트 엔드)
 - React 기반의 SPA(Single Page Application)로 구현하며, Vercel을 통해 호스팅하여 사용자 화면을 제공한다.
2. 서버 사이드 (백엔드)
 - Spring Boot(java) 기반의 RESTful API 서버로 구축하며, AWS EC2 인스턴스에 배포하여 프론트엔드의 요청(주문, 결제, 회원 데이터 등)을 처리한다.
3. 데이터 베이스
 - MySQL을 사용하여 AWS RDS에 구축하며, 상품 정보, 회원 데이터, 커스텀 옵션 주문 내역 등을 안전하게 저장하고 관리한다.

3. 구현할 기능의 우선순위

제한된 개발 기간(7월 15일 마감)을 고려하여 프로토타입 구현 기능을 1순위(MVP)와 2순위(확장)로 나누어 전략적으로 개발한다.

1순위

- 프론트엔드-백엔드-DB 연동 및 기본 아키텍처 세팅
- 커스텀 로스팅 다단계 주문 (Wizard UI) 시스템
- 커피 취향 테스트 및 추천 알고리즘
- 회원가입/로그인 및 비회원 주문 시스템
- 상품 목록 조회, 장바구니, 주문 결제 코어 로직

2순위

- 정기 구독 시스템 (주기별 자동 결제 모델)
- 오프라인 클래스(바리스타, 핸드드립반) 예약 시스템
- 리뷰 작성 및 별점 시스템
- 다국어 지원 및 소셜 로그인(OAuth 2.0) 기능

5. 기타

1. 프로토타입 구현 시 예상 문제점 및 해결 방안

프론트 엔드 - 백엔드 연동 오류 (CORS 문제)

- **문제점:** 과거 유사 프로젝트 진행 시, 독립된 환경(포트)에서 구동되는 React(프론트엔드)와 Spring Boot(백엔드) 간의 API 통신 과정에서 교차 출처 리소스 공유(CORS) 정책 위반 에러가 발생하여 프로젝트 진행이 지연된 경험이 있다.
- **해결 방안:** 이를 원천 차단하기 위해 3단계 사전 방지 전략을 수립한다. 첫째, 프론트엔드 연동 전 Postman을 활용하여 백엔드 API 단독 검증을 100% 완료한다. 둘째, Spring Boot의 `WebMvcConfigurer`를 통해 리액트 개발 서버(`localhost:3000`) 및 향후 상용 배포 도메인에 대한 접근을 명시적으로 허용한다. 셋째, 리액트 개발 환경 내에 Proxy 설정을 추가하여 포트 충돌을 사전에 차단한다.

커스텀 옵션에 따른 데이터 무결성 유지

- **문제점:** 쇼핑몰 운영 중 원두의 기본 가격이나 옵션(용량, 로스팅 등)의 추가 금액이 변동될 수 있다. 이때 과거의 주문 내역 데이터까지 함께 변경되어 영수증 정보가 왜곡되는 데이터 무결성 훼손이 발생할 수 있다.
- **해결 방안:** 주문 상세(`order_items`) 테이블에 상품과 옵션 ID만 외래키로 연결하는 것이 아니라, 주문 발생 시점의 상품명, 옵션명, 결제 단가를 기록하는 '스냅샷(Snapshot)' 필드를 두어 과거 데이터의 불변성을 보장한다.

2. 앞으로의 발전 방향 (2차 고도화 및 상용화)

완벽한 O2O 생태계 구축

- 프로토타입 단계의 온라인 커스텀 원두 판매를 넘어, 오프라인 매장 방문 및 경험 확장을 유도하는 O2O 서비스를 완성한다.
- 2차 개발(5월 30일 이후)을 통해 사용자 취향에 맞춘 '원두 정기 구독 서비스'를 도입하고, 오프라인 로스터리 매장에서 진행되는 '바리스타/핸드드립 클래스 예약 시스템'을 웹사이트 내에 통합 구축한다.

실제 상용 서비스 배포 및 인프라 고도화

- 초기 프로토타입은 로컬(Localhost) 환경에서 테스트를 완료한 후, 실제 카페 고객들이 접속할 수 있도록 클라우드 환경에 정식 배포한다.
- 비용 효율성과 성능을 고려하여 프론트엔드는 Vercel을 통한 자동 배포(CI/CD) 및 HTTPS 환경을 구축하고, 백엔드 및 데이터베이스는 AWS(Amazon Web Services)의 EC2와 RDS를 활용하여 안정적인 실무형 상용 인프라 아키텍처로 고도화한다.